

lab2

資科工碩一 313551027 曹咏萱

Part1.

There are 5 distinct "OFPT_FLOW_MOD" headers during the experiment.

Match fields	Actions	Timeout values
OFPXMT_OMP_ETH_TYPE = Unkown	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = ARP	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = 802.1 LLDP	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_ETH_TYPE = IPv4	Output Port = OFPP_CONTROLLER	0
OFPXMT_OMP_IN_PORT = 1	Output Port = 2	0

The "Unknown" type may indicate an unrecognized message type or a missing handler for a specific OpenFlow version or message.

Part2

1.

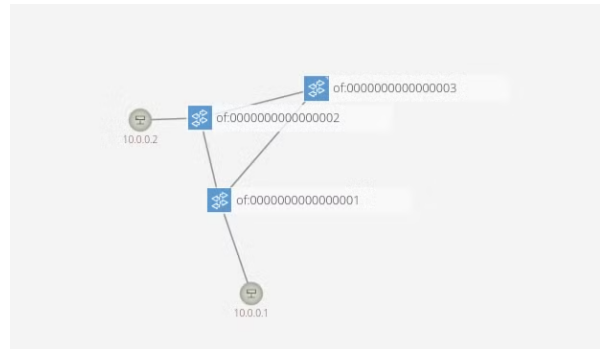
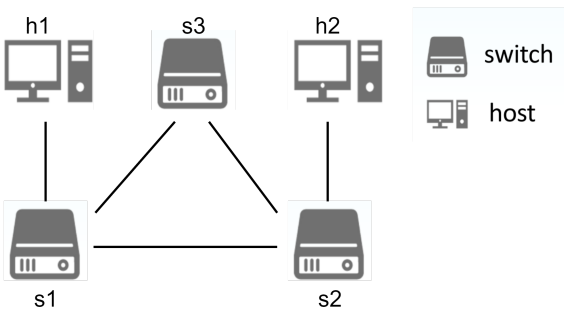
```
reki@SDN-NFV: ~  
reki@SDN-NFV: ~/onos x reki@SDN-NFV: ~/SDN/Lab2 x reki@SDN-NFV: ~ x reki@SDN-NFV: ~/onos x  
*** Starting controller  
c0  
*** Starting 1 switches  
s1 ...  
*** Starting CLI:  
mininet> h1 arping h2  
ARPING 10.0.0.2  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=0 time=8.700 msec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=1 time=4.816 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=2 time=7.963 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=3 time=5.341 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=4 time=3.855 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=5 time=4.618 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=6 time=5.032 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=7 time=9.324 usec  
42 bytes from 2e:c9:32:0b:22:26 (10.0.0.2): index=8 time=4.747 usec  
^C  
--- 10.0.0.2 statistics ---  
9 packets transmitted, 9 packets received, 0% unanswered (0 extra)  
rtt min/avg/max/std-dev = 0.004/0.972/8.700/2.732 ms  
mininet>
```

2.

```
reki@SDN-NFV: ~  
reki@SDN-NFV: ~/onos x reki@SDN-NFV: ~/SDN/Lab2 x reki@SDN-NFV: ~ x reki@SDN-NFV: ~/onos x  
mininet> h1 ping h2  
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.  
^C  
--- 10.0.0.2 ping statistics ---  
58 packets transmitted, 0 received, 100% packet loss, time 59159ms  
  
mininet> h1 ping h2  
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.  
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=70.3 ms  
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.050 ms  
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.050 ms  
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.051 ms  
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.082 ms  
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.052 ms  
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.046 ms  
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.073 ms  
^C  
--- 10.0.0.2 ping statistics ---  
8 packets transmitted, 8 received, 0% packet loss, time 7200ms  
rtt min/avg/max/mdev = 0.046/8.838/70.306/23.232 ms  
mininet>
```

Part3

Create a topology with a loop between switches (h1, h2, h3) to cause broadcast storm.



Install the following flow to each switch and let h1 ping h2.

- Match Fields
 - Ethernet type (ARP)
- Actions
 - Forwarding ARP packets to all port

```
mininet> h1 arping h2 # send ARP request
```

Then, h1 will start sending the packet to s1.

1. Since it is a broadcast packet, s1 copies one and sends it to s2 and s3.
2. When s3 receives the packet it will send it to s3 only, cause it was from s1.
3. At the same time, s2 also receives a packet and starts sending it to s3 and h2.

After the ping, two packets will be retained on the internet. While Layer 2 doesn't have the TTL (Time to Live) mechanism, retained packets will keep increasing and waste a large amount of resources.

The CPU utilization before sending packets.

```

0[|||||] 5.4%] Tasks: 156, 871 thr; 1 running
1[|||||] 7.0%] Load average: 4.29 8.03 6.60
Mem[|||||] 3.63G/6.61G] Uptime: 02:24:35
Swp[|] 1.51M/2.00G]

  PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
 4363 reki        20    0 4784M 1062M 28032 S 11.5 15.7 13:16.66 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
2826 reki        20    0 4182M 311M 158M S  9.8  4.6  7:48.04 /usr/bin/gnome-shell
 896 root         10  -10 295M 41308 5888 S  7.4  0.6  1:51.10 ovs-vswitchd unix:/usr/local/var/run/openvswitch/db.sock
7366 reki        20    0 4784M 1062M 28032 S  7.4 15.7 0:03.31 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
8654 reki        20    0 23228 7808 3712 R  5.7  0.1 0:04.81 htop
3217 reki        20    0 244M 12324 7168 S  1.6  0.2 0:15.73 /usr/bin/ibus-daemon --panel disable
 582 avahi        20    0 7628 4096 3712 S  0.8  0.1 0:00.76 avahi-daemon: running [SDN-NFV.local]
1810 root         20    0 283M 3840 3328 S  0.8  0.1 0:08.22 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.
2852 reki        20    0 4182M 311M 158M S  0.8  4.6 0:14.01 /usr/bin/gnome-shell
3696 reki        20    0 222M 3356 2944 S  0.8  0.0 0:42.13 /usr/bin/VBoxClient --draganddrop
3706 reki        20    0 222M 3484 3072 S  0.8  0.1 0:04.41 /usr/bin/VBoxClient --seamless
3751 reki        20    0 629M 59024 42876 S  0.8  0.9 0:59.52 /usr/libexec/gnome-terminal-server
3957 reki        20    0 4288M 1188M 19328 S  0.8 17.6 0:22.56 bazel(onos) -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpP
4568 reki        20    0 4784M 1062M 28032 S  0.8 15.7 0:35.67 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
4591 reki        20    0 4784M 1062M 28032 S  0.8 15.7 0:06.75 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
4881 reki        20    0 4784M 1062M 28032 S  0.8 15.7 0:14.46 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
7128 reki        20    0 4784M 1062M 28032 S  0.8 15.7 0:04.40 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit

```

The CPU utilization after sending packets.

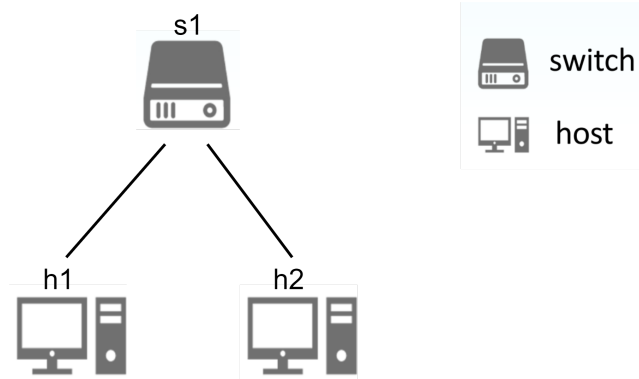
```

0[|||||] 100.0%] Tasks: 157, 873 thr; 2 running
1[|||||] 100.0%] Load average: 3.49 7.67 6.50
Mem[|||||] 3.64G/6.61G] Uptime: 02:24:51
Swp[|] 1.51M/2.00G]

  PID USER      PRI  NI  VIRT   RES   SHR  S  CPU% MEM%   TIME+  Command
 896 root         10  -10 295M 41308 5888 R 200.  0.6  1:52.63 ovs-vswitchd unix:/usr/local/var/run/openvswitch/db.sock
4363 reki        20    0 4784M 1062M 28032 S 112. 15.7 13:17.40 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
2826 reki        20    0 4179M 312M 158M S 100.  4.6  7:50.60 /usr/bin/gnome-shell
3751 reki        20    0 629M 59920 43004 R 62.5  0.9  1:00.77 /usr/libexec/gnome-terminal-server
8876 root         20    0 34284 15616 7424 S 58.3  0.2 0:01.16 /usr/bin/python3 /usr/local/bin/mn --custom=topo_31355102
8654 reki        20    0 23228 7808 3712 R 50.0  0.1 0:05.10 htop
4792 reki        20    0 4784M 1062M 28032 S 37.5 15.7 0:20.94 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
2674 reki        20    0 27248 19100 8064 S 33.3  0.3 0:05.78 /lib/systemd/systemd --user
2856 reki        20    0 4179M 312M 158M S 33.3  4.6 0:01.09 /usr/bin/gnome-shell
4574 reki        20    0 4784M 1062M 28032 S 33.3 15.7 0:04.00 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
8986 nobody        20    0 9936 5888 5504 R 25.0  0.1 0:00.57 arping 10.0.0.2
8946 root         10  -10 295M 41308 5888 S 12.5  0.6 0:00.28 ovs-vswitchd unix:/usr/local/var/run/openvswitch/db.sock
 582 avahi        20    0 7628 4096 3712 S  8.3  0.1 0:00.78 avahi-daemon: running [SDN-NFV.local]
3459 reki        20    0 229M 79952 55324 S  8.3  1.2 1:06.06 /usr/bin/Xwayland :0 -rootless -noreset -accessx -core -a
4881 reki        20    0 4784M 1062M 28032 R  8.3 15.7 0:14.51 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
7366 reki        20    0 4784M 1062M 28032 S  8.3 15.7 0:03.34 /tmp/onos-2.7.0-jdk/bin/java -XX:+UseG1GC -XX:MaxGCPauseM
 572 systemd-o 20    0 14836 6784 6016 S  4.2  0.1 0:18.41 /lib/systemd/systemd-oomd
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9Kill F10Quit

```

Part4



1. [Data Plane] h1 sends the packet to s1.
2. [Data Plane] Since s1 doesn't have a flow rule for forwarding this packet, it generates a PACKET-IN message and sends the packet to the ONOS controller.
3. [Control Plane] Reactive Forwarding Application(org.onosproject.fwd) receives PACKET-IN and executes the function process() to decide the path of forwarding the packet.

```

public void process(PacketContext context){
    ...
    MacAddress macAddress = ethPkt.getSourceMAC();
    ReactiveForwardMetrics macMetrics = null;
    macMetrics = createCounter(macAddress);
    inPacket(macMetrics);
    ...

    // get a set of paths that lead from here to the destination edge switch
    Set<Path> paths =
        topologyService.getPaths(topologyService.currentTopology(),
                                pkt.receivedFrom().deviceId(),
                                dst.location().deviceId());

    if (paths.isEmpty()) {
        // If there are no paths, flood and bail.
        flood(context, macMetrics);
        return;
    }
    ...
}

```

Then, the function installRule() tries to install the rule.

```

// Install a rule forwarding the packet to the specified port.
private void installRule(PacketContext context, PortNumber portNumber,
    ReactiveForwardMetrics macMetrics) {
    ...
    // If PacketOutOnly or ARP packet than forward directly to output port
    if (packetOutOnly || inPkt.getEtherType() == Ethernet.TYPE_ARP) {
        packetOut(context, portNumber, macMetrics);
        return;
    }
    ...
    flowObjectiveService.forward(context.inPacket().receivedFrom().deviceId(),
        forwardingObjective);
    forwardPacket(macMetrics);
}

```

4. [Control Plane] Controller sends a FLOW_MOD to s1 and installs the flow rule.
5. [Control Plane] Controller sends the packet(original) back to s1 with a PACKET-OUT message.
6. [Data Plane] s1 receives FLOW-MOD from the controller and installs the new flow rule in its flow table.
7. [Data Plane] s1 forwards the packet(original) to h2.
8. [Data Plane] After h2 receives the packet, it sends a reply(packet) to h1.
9. [Data Plane] h2 sends the packet to s1. Since s1 also doesn't have a flow rule for forwarding this packet, it generates a PACKET-IN message and sends the packet to the ONOS controller.
10. [Control Plane] Reactive Forwarding Application(org.onosproject.fwd) receives PACKET-IN and executes the function process() to decide the path of forwarding the packet.
11. [Control Plane] Controller sends a FLOW_MOD to s1 and installs the flow rule.
12. [Control Plane] Controller sends the packet(reply) back to s1 with a PACKET-OUT message.
13. [Data Plane] s1 receives FLOW-MOD from the controller and installs the new flow rule in its flow table.
14. [Data Plane] s1 forwards the packet(reply) to h1.

Part5

一開始在使用 curl 時沒辦法成功並會跳出下面的錯誤訊息

```
bash: ' http://localhost:8181/onos/v1/flows/of:0000000000000001 ' no such file or directory
```

後來嘗試過很多方法例如安裝 Java、修改 json 檔等，後來發現似乎是在 windows 將 url 複製貼上到 ubuntu terminal 時，格式會跑掉，於是重新手打 url 之後問題就解決了。

另外，我也在這次實驗中了解何謂 Broadcast Storm 以及該如何避免這個問題，同時也對怎麼使用 Wireshark 有了更深的了解。

References

Day 15 連結層攻擊實作- STP Spoofing - | iT 邦幫忙

<https://ithelp.ithome.com.tw/articles/10247179>

Flow Rules | ONOS Wiki

<https://wiki.onosproject.org/display/ONOS/Flow+Rules>

OpenFlow | Muzixing

<https://www.muzixing.com/pages/2013/12/12/yuan-chuang-openflowtong-xin-liu-cheng-jie-du.html>

apps/fwd/src/main/java/org/onosproject/fwd/ReactiveForwarding.java | GitHub

<https://github.com/opennetworkinglab/onos/blob/onos-2.2/apps/fwd/src/main/java/org/onosproject/fwd/ReactiveForwarding.java>